National University of Computer and Emerging Sciences, Lahore Campus

	Course:	COAL	**Course Code:**	EE2003
	Program:	BS (DS & CS)	**Semester:**	Fall 2026
	Due Date:	**October 19, 2026**	**Total Marks:**	100
	Section:	BCS-3H, BDS-3A,BDS-3B	**Weight:**	
	Assignment:	3 [CH # 5,6,7]	**Page(s):**	5

Instructions/Notes:

- *Q1 must be solved **by hand** and submitted as a scanned copy. The handwritten copy must also be submitted in class.*
- Q2 must be implemented in an Assembly .asm file. Attach **a screenshot** of the successfully executed code in DOSBox along with the **source file** on GCR.
- In case of **plagiarism**, **all assignments and quizzes will be marked zero**.
- In case of **cheating** from any **AI tool**, **all quizzes will be marked zero**.
- *Mention your roll no., name, and section properly while submitting.*

Q1. Answer the Following Questions [25 Marks]

(a) Display Memory Mapping & Video Attributes [5 Marks]

i. An 8088 program writes a word directly to logical address `[AF00:994A]`. Given that text-mode display memory is physically mapped starting at `0xB8000`, determine the physical address accessed and find its corresponding display coordinate in terms of (Row, Column) on an 80×25 display screen. **Show complete working.**

ii. What values should be loaded into ES, DI, and AX to print the character 'V' (ASCII = `0x56`) at Row 14, Column 35 with **Blinking High-Intensity Yellow** foreground on a **Blue** background using `mov word [es:di], ax`? Provide both the **binary and hexadecimal** representation of the **attribute byte**.

(b) Subroutines & Stack Frame Architecture [5 Marks]

i. A subroutine `ProcessData` receives 3 word parameters via the stack pushed in caller order (arg1, arg2, arg3), allocates 2 word-sized local variables (var1, var2) on the stack, and preserves caller registers AX, BX, and SI. Write the complete assembly **prologue and epilogue** using **BP** as the frame pointer, ensuring **callee parameter cleanup**.

ii. Assuming initial SP = `0xFFFE` prior to pushing arguments:

- What is the value of SP immediately after the subroutine's prologue finishes executing?
- What is the value of SP immediately after the subroutine returns to the caller?

(c) String Primitives Trace & Flag Behavior [5 Marks]

Given the following data declarations and assembly code fragment:

```
[org 0x0100]
jmp start
src:  db 'FAST-NU-LHR', 0
dest: db 'FAST-NUCES', 0
start:
    push cs
    pop ds
    push cs
    pop es
    mov si, src     ; Assume offset of src is 0x0102
    mov di, dest    ; Assume offset of dest is 0x010E
    mov cx, 11
    cld
    repe cmpsb
```

i. State the final contents of **SI, DI, and CX (in hex)** after `repe cmpsb` halts execution.

ii. Determine the final status of the **Zero Flag (ZF)** and **Carry Flag (CF)**.

iii. What would happen to the comparison behavior if `cld` were replaced by `std`?

(d) Video Memory Output Simulation [5 Marks]

Assuming the video screen is cleared prior to execution, analyze the following code and determine the **exact visual output** produced on the display screen:

```
[org 0x0100]
mov ax, 0xb800
mov es, ax
xor di, di
mov cx, 5
mov dx, 0x0C30
loop_draw:
    mov ax, cx
    shl ax, 1
    add al, 0x30
    mov dl, al
    mov word [es:di], dx
    add di, 320
    loop loop_draw
mov ax, 0x4c00
int 0x21
```

Complete the trace table below:

Iteration	CX	Character Printed	Video Offset (DI)	Row	Column
1					
2					
3					
4					
5					

(e) Architectural Equivalences & Stack Manipulation [5 Marks]

i. Replace each of the following illegal or compound operations with a **single valid 8088 instruction** that produces the **exact same architectural effect**:

- `pop ip`
- `sub sp, 2` followed by `mov [ss:sp], dx`
- `mov ax, [ss:sp]` followed by `add sp, 2`
- `add sp, 6` followed by `mov ip, [ss:sp-6]`

ii. Write an assembly sequence to swap the values of registers AX and BX using **only stack operations**. You are **strictly forbidden** from using temporary general-purpose registers or memory variables.

Q2 Text Viewport Manager with Windowed Scrolling & Live Keyword Highlighter [60 Marks]

In text-mode word processors and terminals, viewing documents larger than the screen requires a bounded viewport window that can scroll its internal text lines independently without corrupting outer UI borders, alongside a real-time keyword highlighter that dynamically marks matching search patterns directly inside video memory.

In this assignment, you will write a modular 16-bit 8088 Assembly Language program that constructs a dedicated windowed text viewer on the screen (`0xB800`), renders a multi-line document buffer, scrolls text **strictly within the defined window boundaries** using 2D block movement, and locates and highlights all occurrences of a search keyword across the display area using string comparison instructions.

System Requirements & Architectural Specifications

1. UI Frame & Viewport Initialization

- **Clear the entire 80 × 25 screen** with blanks using **string instructions**.
- Construct an outer viewport window spanning from row 3 to row 21 and column 8 to column 72 using double-line extended ASCII border characters (`0xC9`, `0xBB`, `0xBA`, `0xCD`, `0xC8`, `0xBC`).

- Display a title banner centered at the top of the window with attribute `0x1E` (Yellow on Blue).

2. Multi-line Text Buffer Streaming

- Load and display a paragraph of text stored in memory line-by-line inside the window boundaries (margins: left column +2, right column −2).
- Use string primitives (`LODSB/STOSW`) to stream characters into the video segment.

3. Bounded Sub-Window Scrolling (Window Scroll Up)

- Implement a subroutine `scroll_window_up` that receives the window coordinates (top, left, bottom, right) via the stack.
- Unlike full-screen scrolling, this routine must shift **only the text inside the window** upward by N lines **without altering or corrupting the window borders** or external screen areas.
- Move lines using `MOVSW` with row-stride offset calculations for each individual line, and blank out the vacated bottom line(s) of the window using `STOSW`.

4. In-Memory Substring Search & Highlighting Engine

- Implement a subroutine `find_and_highlight` that receives the search substring offset and target highlight attribute (e.g., `0xCE` – Blinking Yellow on Red) via the stack.
- Traverse the display memory within the active viewport using string instructions (`REPE CMPSB` / `SCASB`) to match consecutive characters.
- For every match found on the screen, overwrite its corresponding video attribute bytes to highlight the keyword **without modifying the underlying ASCII characters**.

5. Statistics Status Bar (Bottom Row)

- On row 23, display a summary status bar formatted as:

```
Total Matches Found: [Count] | Viewport Status: ACTIVE
```

- The integer count must be printed using a dedicated **recursive or iterative stack-based** `printNum` subroutine.

Subroutine Architecture & Calling Convention Rules

- Every procedure must receive its parameters **strictly through the stack**.
- Frame access must be standardized using **BP relative addressing** (`[bp + offset]`).
- Subroutines must allocate any temporary counters or accumulators as **local variables on the stack** (`sub sp, N` and `[bp - offset]`).
- All modified registers must be **preserved and restored** via `push/pop`.
- Callee stack cleanup must be **strictly enforced** using the `ret n` instruction.

Version Control (GitHub Repository) [15 Marks]

- Maintain one GitHub repository for your COAL / Assembly Language work. You may name it anything (e.g., COAL, Assembly-Programs).
- All .asm source files of your assignments must be pushed to this repository, with each assignment in its own folder (e.g., Assignment3/).
- Use regular, meaningful commits with descriptive messages (e.g., "Added draw_window subroutine") to show your version control history.
- The repository must contain a proper README.md that includes: the repository title, your name and roll no., a short description of the programs, the tools used (NASM, DOSBox), how to assemble and run the programs, the folder structure, and output screenshots.
- Write the repository link on the first page of your submission.
- Commits made after the deadline will not be considered.

Marking Breakdown (Total: 75 Marks)

Assessment Criteria	Marks
Screen Initialization & Bordered Frame Generation Subroutine (`draw_window`)	10
Text Streaming inside Bounded Margins via String Primitives (LODSB/STOSW)	10
Bounded Sub-Window Scrolling Engine (Row Strides + MOVSW + STOSW)	15
Substring Search & In-Place Attribute Highlighting Engine (REPE CMPSB)	15
Numeric Output Subroutine (`printNum`) & Status Bar Formatting	5
Stack Frame Discipline (BP), Local Variables & Callee Stack Cleanup (`ret n`)	5
GitHub Repository: all .asm files pushed, commit history, proper README.md	15
Total	**75**